

4.7 SUMMARY

When seen from the outside, a file system is a collection of files and directories, plus operations on them. Files can be read and written, directories can be created and destroyed, and files can be moved from directory to directory. Most modern file systems support a hierarchical directory system in which directories may have subdirectories and these may have subsubdirectories ad infinitum.

When seen from the inside, a file system looks quite different. The file system designers have to be concerned with how storage is allocated and how the system keeps track of which block goes with which file. Possibilities include contiguous files, linked lists, file-allocation tables, and i-nodes. Different systems have different directory structures. Attributes can go in the directories or somewhere else (e.g., an i-node). Disk space can be managed using free lists or bitmaps. File-system reliability is enhanced by making incremental dumps and by having a program that can repair sick file systems. File-system performance is important and can be enhanced in several ways, including caching, read ahead, and carefully placing the blocks of a file close to each other. Log-structured file systems also improve performance by doing writes in large units.

Examples of file systems include ISO 9660, MS-DOS, and UNIX. These differ in many ways, including how they keep track of which blocks go with which file, directory structure, and management of free disk space.

PROBLEMS

1. In Windows, when a user double clicks on a file listed by Windows Explorer, a program is run and given that file as a parameter. List two different ways the operating system could know which program to run.
2. In early UNIX systems, executable files (*a.out* files) began with a very specific magic number, not one chosen at random. These files began with a header, followed by the text and data segments. Why do you think a very specific number was chosen for executable files, whereas other file types had a more-or-less random magic number as the first word?
3. In Fig. 4-5, one of the attributes is the record length. Why does the operating system ever care about this?
4. Is the `open` system call in UNIX absolutely essential? What would the consequences be of not having it?
5. Systems that support sequential files always have an operation to rewind files. Do systems that support random-access files need this, too?
6. Some operating systems provide a system call `rename` to give a file a new name. Is there any difference at all between using this call to rename a file and just copying the file to a new file with the new name, followed by deleting the old one?

7. A simple operating system supports only a single directory but allows it to have arbitrarily many files with arbitrarily long file names. Can something approximating a hierarchical file system be simulated? How?
8. In UNIX and Windows, random access is done by having a special system call that moves the “current position” pointer associated with a file to a given byte in the file. Propose an alternative way to do random access without having this system call.
9. Consider the directory tree of Fig. 4-9. If */usr/jim* is the working directory, what is the absolute path name for the file whose relative path name is *../ast/x*?
10. Contiguous allocation of files leads to disk fragmentation, as mentioned in the text, because some space in the last disk block will be wasted in files whose length is not an integral number of blocks. Is this internal fragmentation or external fragmentation? Make an analogy with something discussed in the previous chapter.
11. Suppose a filesystem check reveals that a block has been allocated to two different files, */home/hjb/dadjokes.txt* and */etc/motd*. Both are text files. The filesystem check duplicates the block’s data and re-assigns */etc/motd* to use the new block. Answer the following questions. (i) In what realistic circumstance(s) could the data from both files still remain correct and consistent with their original content? (ii) How might the user investigate whether the files have been corrupted? (iii) If one or both of the files’ data have been corrupted, what mechanisms might allow the user to recover the data?
12. One way to use contiguous allocation of the disk and not suffer from holes is to compact the disk every time a file is removed. Since all files are contiguous, copying a file requires a seek and rotational delay to read the file, followed by the transfer at full speed. Writing the file back requires the same work. Assuming a seek time of 5 msec, a rotational delay of 4 msec, a transfer rate of 8 MB/sec, and an average file size of 8 KB, how long does it take to read a file into main memory and then write it back to the disk at a new location? Using these numbers, how long would it take to compact half of a 16-GB disk?
13. MacOS has symbolic links and also aliases. An alias is similar to a symbolic link; however, unlike symbolic links, an alias stores additional metadata about the target file (such as its inode number and file size) so that, if the target file is moved within the same filesystem, accessing the alias will result in accessing the target file, as the filesystem will search for and find the original target. How could this behavior be beneficial compared to symbolic links? How could it cause problems?
14. Following on the previous question, in earlier MacOS versions, if the target file is moved and then another file is created with the original path of the target, the alias would still find and use the moved target file (not the new file with the same path/name). However, in versions of MacOS 10.2 or later, if the target file is moved and another is created in the old location, the alias will connect to the new file. Does this address the drawbacks from your answer to the previous question? Does it dampen the benefits you noted?
15. Some digital consumer devices need to store data, for example as files. Name a modern device that requires file storage and for which contiguous allocation would be a fine idea.

16. Consider the i-node shown in Fig. 4-15. If it contains 10 direct addresses of 4 bytes each and all disk blocks are 1024 KB, what is the largest possible file?
17. For a given class, the student records are stored in a file. The records are randomly accessed and updated. Assume that each student's record is of fixed size. Which of the three allocation schemes (contiguous, linked and table/indexed) will be most appropriate?
18. Consider a file whose size varies between 4 KB and 4 MB during its lifetime. Which of the three allocation schemes (contiguous, linked and table/indexed) will be most appropriate?
19. It has been suggested that efficiency could be improved and disk space saved by storing the data of a short file within the i-node. For the i-node of Fig. 4-15, how many bytes of data could be stored inside the i-node?
20. Two computer science students, Carolyn and Elinor, are having a discussion about i-nodes. Carolyn maintains that memories have gotten so large and so cheap that when a file is opened, it is simpler and faster just to fetch a new copy of the i-node into the i-node table, rather than search the entire table to see if it is already there. Elinor disagrees. Who is right?
21. Name one advantage of hard links over symbolic links and one advantage of symbolic links over hard links.
22. Explain how hard links and soft links differ with respect to i-node allocations.
23. Consider a 4-TB disk that uses 8-KB blocks and the free-list method. How many block addresses can be stored in one block?
24. Free disk space can be kept track of using a free list or a bitmap. Disk addresses require D bits. For a disk with B blocks, F of which are free, state the condition under which the free list uses less space than the bitmap. For D having the value 16 bits, express your answer as a percentage of the disk space that must be free.
25. The beginning of a free-space bitmap looks like this after the disk partition is first formatted: 1000 0000 0000 0000 (the first block is used by the root directory). The system always searches for free blocks starting at the lowest-numbered block, so after writing file A , which uses six blocks, the bitmap looks like this: 1111 1110 0000 0000. Show the bitmap after each of the following additional actions:
 - (a) File B is written, using five blocks.
 - (b) File A is deleted.
 - (c) File C is written, using eight blocks.
 - (d) File B is deleted.
26. What would happen if the bitmap or free list containing the information about free disk blocks was completely lost due to a crash? Is there any way to recover from this disaster, or is it bye-bye disk? Discuss your answers for UNIX and the FAT-16 file system separately.
27. Oliver Owl's night job at the university computing center is to change the tapes used for overnight data backups. While waiting for each tape to complete, he works on writing his thesis that proves Shakespeare's plays were written by extraterrestrial visitors.

His text processor runs on the system being backed up since that is the only one they have. Is there a problem with this arrangement?

28. We discussed making incremental dumps in some detail in the text. In Windows it is easy to tell when to dump a file because every file has an archive bit. This bit is missing in UNIX. How do UNIX backup programs know which files to dump?
29. Suppose that file 21 in Fig. 4-27 was not modified since the last dump. In what way would the four bitmaps of Fig. 4-28 be different?
30. It has been suggested that the first part of each UNIX file be kept in the same disk block as its i-node. What good would this do?
31. Consider Fig. 4-29. Is it possible that for some particular block number the counters in *both* lists have the value 2? How should this problem be corrected?
32. The performance of a file system depends upon the cache hit rate (fraction of blocks found in the cache). If it takes 1 msec to satisfy a request from the cache, but 40 msec to satisfy a request if a disk read is needed, give a formula for the mean time required to satisfy a request if the hit rate is h . Plot this function for values of h varying from 0 to 1.0.
33. For an external USB hard drive attached to a computer, which is more suitable: a write-through cache or a block cache?
34. Consider an application where students' records are stored in a file. The application takes a student ID as input and subsequently reads, updates, and writes the corresponding student record; this is repeated till the application quits. Would the "block read-ahead" technique be useful here?
35. Discuss the design issues involved in selecting the appropriate block size for a file system.
36. Consider a disk that has 10 data blocks starting from block 14 through 23. Let there be 2 files on the disk: f1 and f2. The directory structure lists that the first data blocks of f1 and f2 are, respectively, 22 and 16. Given the FAT table entries as below, what are the data blocks allotted to f1 and f2?

(14,18); (15,17); (16,23); (17,21); (18,20); (19,15); (20, -1); (21, -1); (22,19); (23,14).

In the above notation, (x, y) indicates that the value stored in table entry x points to data block y .
37. In the text, we discussed two major ways to identify file type: file extensions and investigation of file content (e.g., by using headers and magic numbers). Many modern UNIX filesystems support extended attributes which can store additional metadata for a file, including file type. This data is stored as part of the file's attribute data (in the same way that file size and permissions are stored). How is the extended attribute approach for storing files better or worse than the file extension approach or identifying file type by content?
38. Consider the idea behind Fig. 4-23, but now for a disk with a mean seek time of 8 msec, a rotational rate of 15,000 rpm, and 262,144 bytes per track. What are the data rates for block sizes of 1 KB, 2 KB, and 4 KB, respectively?

39. In this chapter, we have seen that SSDs do their best to avoid writing the same memory cells frequently (because of the wear). However, many SSDs offer much more functionality than what we presented so far. For instance, many controllers implement compression. Explain why compression may help with reducing the wear.
40. Given a disk-block size of 4 KB and block-pointer address value of 4 bytes, what is the largest file size (in bytes) that can be accessed using 11 direct addresses and one indirect block?
41. The MS-DOS FAT-16 table contains 64K entries. Suppose that one of the bits had been needed for some other purpose and that the table contained exactly 32,768 entries instead. With no other changes, what would the largest MS-DOS file have been under this condition?
42. Files in MS-DOS have to compete for space in the FAT-16 table in memory. If one file uses k entries, that is k entries that are not available to any other file, what constraint does this place on the total length of all files combined?
43. How many disk operations are needed to fetch the i-node for a file with the path name */usr/ast/courses/os/handout.t*? Assume that the i-node for the root directory is in memory, but nothing else along the path is in memory. Also assume that all directories fit in one disk block.
44. In many UNIX systems, the i-nodes are kept at the start of the disk. An alternative design is to allocate an i-node when a file is created and put the i-node at the start of the first block of the file. Discuss the pros and cons of this alternative.
45. Write a program that reverses the bytes of a file, so that the last byte is now first and the first byte is now last. It must work with an arbitrarily long file, but try to make it reasonably efficient.
46. Write a program that starts at a given directory and descends the file tree from that point recording the sizes of all the files it finds. When it is all done, it should print a histogram of the file sizes using a bin width specified as a parameter (e.g., with 1024, file sizes of 0 to 1023 go in one bin, 1024 to 2047 go in the next bin, etc.).
47. Write a program that scans all directories in a UNIX file system and finds and locates all i-nodes with a hard link count of two or more. For each such file, it lists together all file names that point to the file.
48. Write a new version of the UNIX *ls* program. This version takes as an argument one or more directory names and for each directory lists all the files in that directory, one line per file. Each field should be formatted in a reasonable way given its type. List only the first disk address, if any.
49. Implement a program to measure the impact of application-level buffer sizes on read time. This involves writing to and reading from a large file (say, 2 GB). Vary the application buffer size (say, from 64 bytes to 4 KB). Use timing measurement routines (such as *gettimeofday* and *getitimer* on UNIX) to measure the time taken for different buffer sizes. Analyze the results and report your findings: does buffer size make a difference to the overall write time and per-write time?

50. Implement a simulated file system that will be fully contained in a single regular file stored on the disk. This disk file will contain directories, i-nodes, free-block information, file data blocks, etc. Choose appropriate algorithms for maintaining free-block information and for allocating data blocks (contiguous, indexed, linked). Your program will accept system commands from the user to perform file system operations, including at least one to create/delete directories, create/delete/open files, read/write from/to a selected file, and to list directory contents.